

csiro-biomass

8th Place Solution | PixelScale

Rank	8
Team	PixelScale
Author	Paritosh Kumar Tripathi
Topic ID	671363
Version	Original

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/671363>

Retrieved with Kaggle CLI on 2026-07-18.

1. Overview & Key Insight

Our solution is a weighted ensemble of three distinct pipelines, all anchored by a **DINOv3 Huge** backbone.

Our final submission combined:

1. **SSA-Adapted Pipeline (50%)**: Test-Time Adaptation using subspace alignment.
2. **Species-Aware Pipeline (30%)**: Two-stage injection of species metadata.
3. **Dual-Backbone Pipeline (20%)**: DINOv3 fused with SigLIP features.

2. The Core Backbone (Takumi's Part)

- I strongly believed in the trend of scaling laws and aimed to utilize the largest model possible within my compute constraints.
- A key insight was that in competitions with limited data, utilizing a strong backbone pretrained on a massive dataset is crucial.
- I spent the majority of the competition optimizing the training recipe for DINOv3, which I believe gave me an edge over other solutions using the same architecture.
- I observed a significant jump in the Public LB score as I increased the model size. Increasing the model size proved to be more effective than increasing the image resolution. Due to A100 memory constraints, we standardized on:
 - **Model**: `vit_huge_patch16_dinov3.lvd1689m`
 - **Image Size**: 768px (Standardized across most pipelines)

I wanted to emphasize the impact of simply changing the backbone. Here is the progression:

- `convnext_tiny` → `vit_base_patch16_dinov3.lvd1689m`: 0.61 → 0.67
- `vit_base_patch16_dinov3.lvd1689m` → `vit_large_patch16_dinov3.lvd1689m`: 0.69 → 0.72
- `vit_large_patch16_dinov3.lvd1689m` → `vit_huge_patch16_dinov3.lvd1689m`: 0.72 → 0.74

Head Architecture

I utilized a standard regression head with `SiLU` activation and dropout.

```
nn.Sequential(  
    nn.Linear(in_features, hidden),  
    nn.SiLU(),  
    nn.Dropout(0.3),  
    nn.Linear(hidden, out_features),  
)
```

Training Strategy

To stabilize the training of such a large model on limited data, I employed a **Two-Stage Training** process using AdamW and Mixed Precision (AMP).

- **Loss Function:** `nn.SmoothL1Loss()`
- **Batch Size:** Effective batch size of 8 (using `grad_accum_steps=8` on Batch Size 1).

Stage 1: Head Only (Warmup)

- **Goal:** Stabilize weights before fine-tuning the transformer.
- **Epochs:** 15
- **LR:** `3e-4` with `ReduceLR0nPlateau`.

Stage 2: Full Backbone Finetuning

- **Goal:** Adapt the huge backbone without overfitting.
- **Epochs:** 35
- **Weight Decay:** `0.2` (High decay was crucial for regularization).
- **Scheduler:** Cosine with a strict warmup phase.

```
# Stage 2 Scheduler Configuration
CosineLRScheduler(
    lr_min=1e-6,          # Target minimum LR
    warmup_t=3,           # Crucial 3-epoch warmup
    warmup_lr_init=1e-7,  # Start near zero
    cycle_limit=1,        # Single cycle (no restarts)
    t_in_epochs=True
)
```

Data Augmentation

I used **Albumentations** to force the model to learn invariant features, specifically using `RandomShadow` and `RandomGamma` to handle lighting variations.

```
A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomRotate90(p=0.5),
    A.ColorJitter(brightness=0.2, contrast=0.1, saturation=0.2, p=0.5),
    A.RandomGamma(gamma_limit=(80, 120), p=0.3),
    A.RandomShadow(num_shadows_limit=(1, 3), shadow_dimension=5, p=0.5),
    A.Resize(self.img_size, self.img_size),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])
```

Validation Strategy

I initially tried **GroupKFold** using `sampling_date`, but it led to high distribution variance between folds, and some models failed to learn sufficient data variation. This did not improve the Public LB.

Therefore, I decided to accept the discrepancy between CV and Public LB and maximized the CV score using a stratified approach based on bins and species.

```
# Create a stratification key combining total biomass bin and species
df['strat_key'] = df['total_bin'].astype(int) * 100 + df['species_id'].astype(int)

skf = StratifiedKFold(
    n_splits=self.config.n_folds,
    shuffle=True,
    random_state=self.config.random_state
)
```

Species-Aware Prediction

To utilize the species metadata effectively during inference (where metadata might not be available or reliable):

1. **Species Model** Trained a separate `dinov3_large` model (same head structure) to classify species from images.
2. **Inference**
 - Predicted the probability of the image belonging to each species using the Species Model.
 - Used this predicted probability as an input feature for the main Biomass Prediction model.

3. Critical Optimizations (Diptyajit Das's Part)

While the backbone provided the power, Diptyajit's experiments with targets and resolution provided the stability and generalization needed for the Private LB.

A. Target Reformulation

Instead of predicting the standard targets directly, we reformulated the regression problem to align better with the biological components.

- **Old Targets:** `Dry_Green_g`, `GDM_g`, `Dry_Total_g`
- **New Targets:** `Dry_Clover_g`, `GDM_g`, `Dry_Total_g`
- **Impact:** This specific change was crucial for the **Huge DINOv3** variant, directly improving the Public LB from **0.73 to 0.74**.

B. Auxiliary Heads & Stability

I experimented with adding an auxiliary NDVI regression head alongside species classification in Stage 1. This boosted the `vit_large` model to 0.73, but had negligible impact on `vit_huge_plus` model's score.

C. The "Golden" Resolution (640px)

While 768px provided the **best Public LB**, Diptyajit's experiments revealed that **640px** offered superior generalization and resulted in our **highest single-model Private LB (0.657)**, but we couldn't select this for final evaluation because of lower Public LB (0.75)

4. Pipeline A: The SSA Adapter & MLP Head (Paritosh's Part)

Weight in Ensemble: 0.5

This pipeline addressed the domain shift between training and test sets without expensive retraining. Instead of standard fine-tuning, we used **Significant Subspace Alignment (SSA)** as a lightweight Test-Time Adaptation (TTA) technique.

The "2-Layer" MLP Head

- I replaced the standard linear head with a deeper projection head, which gave a +0.02 boost from Public LB **0.74 to 0.76** and +0.01 improvement from **0.635 to 0.646** in Private LB.
- I then finalized upon 3-Layer MLP Head because it was slightly better than 2-Layers.

```
nn.Sequential(  
    nn.Linear(input_dim, 512), # Hidden Layer 1  
    nn.BatchNorm1d(512),  
    nn.ReLU(),  
    nn.Dropout(0.3),  
  
    nn.Linear(512, 256),      # Hidden Layer 2  
    nn.BatchNorm1d(256),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
  
    nn.Linear(256, 128),     # Hidden Layer 3  
    nn.BatchNorm1d(128),  
    nn.ReLU(),  
    nn.Dropout(0.2),  
  
    nn.Linear(128, n_targets)  
)
```

SSA Adapter (Test-Time Adaptation)

I inserted a learnable linear adapter between the backbone and the head. During inference, we froze the backbone and updated only this adapter to align the test image features with the training distribution (using pre-calculated PCA statistics `mu_s` and `U`). More details here: <https://arxiv.org/pdf/2410.03263>

This had negligible change in Public LB but I trusted this approach and it improved the score from **0.646 to 0.652** in Private LB :)

5. Pipeline B: Species-Aware Injection (Takumi's Part)

Weight in Ensemble: 0.3

1. **Stage 1 (Species Model):** Trained `vit_large_patch16_dinov3` to classify 15 species.
 2. **Stage 2 (Biomass Model):**
 - Backbone: `vit_huge_plus_patch16_dinov3`.
 - Input: Concatenates image features + **Predicted Species Probabilities**.
 - Targets: Predicted `Dry_Clover_g` and `GDM_g` to reconstruct the total.
-

6. Pipeline C: Dual-Backbone Fusion with SigLIP (Takumi's Part)

Weight in Ensemble: 0.2

To capture different semantic granularities, we fused features from two state-of-the-art vision models.

- **Branch 1:** DINOv3 Huge (768px) - Excellent for structure and local details.
- **Branch 2:** SigLIP `vit_so400m_patch16_siglip_512` (512px) - Excellent for semantic understanding.
- **Fusion:** Features from both backbones were concatenated before passing to the MLP head.

```
# Feature Fusion Logic
feat_dino = dino_backbone(img_768) # DINOv3 Features
feat_siglip = siglip_backbone(img_512) # SigLIP Features

# Concatenate: [Backbone_L, Backbone_R, SigLIP_L, SigLIP_R]
combined_features = torch.cat([feat_dino, feat_siglip], dim=1)
```

7. Final Ensemble & Results

Our final submission was a weighted average of the three pipelines. The heavy weighting on the SSA model (0.5) reflects its robustness to the distribution shift in the private test set. The final ensemble helped gained the last few decimals.

```
# Final Ensemble Logic
final_submission = (
    0.5 * prediction_ssa +      # Pipeline A (Robustness)
    0.3 * prediction_species +  # Pipeline B (Metadata Context)
    0.2 * prediction_siglip     # Pipeline C (Feature Diversity)
)
```

Validation Strategy

We used **StratifiedKFold (5 Folds)**, stratifying by a composite key of `total_biomass_bin * species_id` to ensure every fold saw a representative distribution of species and weight classes.

Performance

- **Public LB:** 0.760
- **Private LB:** 0.652

Supplemental Q&A

minghigh · 2026-02-03T02:49:20.863000

Congratulations !

A. Target Reformulation Instead of predicting the standard targets directly, we reformulated the regression problem to align better with the biological components. Old Targets: Dry_Green_g, GDM_g, Dry_Total_g New Targets: Dry_Clover_g, GDM_g, Dry_Total_g Impact: This specific change was crucial for the Huge DINOv3 variant, directly improving the Public LB from 0.73 to 0.74.

The meaning is that use 'Dry_Clover_g', 'GDM_g', and 'Dry_Total_g' will perform better than 'Dry_Green_g', 'GDM_g', and 'Dry_Total_g'. Is this the case for all of your models?

Paritosh Kumar Tripathi · 2026-02-04T12:52:56.233000

We did not test it on all of our models, but for dino v3 huge there was indeed improvements in both CV and LB. We focused on diversity so my MLP heads predicted all 5 targets (Pipeline A), whereas Pipeline B and C predicted 3 targets and recreated the 5 from these 3.